ESAME DI JAVA · 11/11/2026

Cognome e nome _____ Punti _____ / 30,5

Parte 1 · Teoria (9 punti)

Una sola risposta corretta per domanda. Corretta **+0,6**, sbagliata **–0,15**, non data **0**

1. *Quale affermazione su un metodo* **`static`** *è vera?*
   - A. Può leggere i campi dell'oggetto con **`this`**
   - B. Appartiene alla classe e si può chiamare senza creare un oggetto
   - C. Si può chiamare solo dopo aver creato almeno un oggetto
   - D. Ne esiste una copia per ogni oggetto
   - E. Nessuna delle precedenti

2. *Quando una classe ha il costruttore senza parametri generato dal compilatore?*
   - A. Sempre
   - B. Solo se tutti i campi sono primitivi
   - C. Solo se la classe non dichiara nessun costruttore
   - D. Solo se la classe dichiara un costruttore con parametri
   - E. Nessuna delle precedenti

3. *A che cosa serve dichiarare* **`private`** *un campo e offrire getter e setter?*
   - A. A rendere il programma più veloce
   - B. A far sì che il campo sia condiviso da tutti gli oggetti
   - C. A impedire che il campo cambi valore dopo il costruttore
   - D. A far passare ogni accesso dai metodi della classe, che possono validare i dati
   - E. Nessuna delle precedenti

4. *Una classe ha i campi* **`int n`**, **`boolean b`** *e* **`String s`**, *mai assegnati. Che valori hanno in un oggetto appena creato?*
   - A. **`0`**, **`false`**, **`null`**
   - B. **`0`**, **`true`**, **`null`**
   - C. **`0`**, **`false`**, **`""`**
   - D. Non hanno valore: il codice non compila
   - E. Nessuna delle precedenti

5. *Che cosa riceve un metodo quando gli si passa una variabile di tipo classe?*
   - A. Una copia dell'oggetto
   - B. La variabile originale, che il metodo può far puntare a un altro oggetto
   - C. Solo i campi pubblici dell'oggetto
   - D. Una copia del riferimento: stesso oggetto, variabile diversa
   - E. Nessuna delle precedenti

6. *Un metodo esegue* **`Studente s = new Studente("Anna")`**. *Dove si trovano* **`s`** *e l'oggetto?*
   - A. **`s`** nello stack, l'oggetto nello heap
   - B. Entrambi nello stack
   - C. Entrambi nello heap
   - D. **`s`** nello heap, l'oggetto nello stack
   - E. Nessuna delle precedenti

7. *Quando un oggetto può essere eliminato dal garbage collector?*
   - A. Appena termina il metodo che l'ha creato
   - B. Quando si chiama il suo costruttore una seconda volta
   - C. Quando nessun riferimento raggiungibile lo indica più
   - D. Solo quando il programma termina
   - E. Nessuna delle precedenti

8. *Che cosa significa che* **`String`** *è immutabile?*
    A. Una variabile **`String`** non può essere riassegnata
    B. Le stringhe non si possono concatenare
    C. Un oggetto **`String`** non cambia: le operazioni di modifica producono un nuovo oggetto
    D. Una stringa non può essere **`null`**
    E. Nessuna delle precedenti

9. *Una classe inizia con* **`package modello`** *e i sorgenti del progetto stanno in* **`src`**. *Dove va il suo file?*
    A. Direttamente in **`src`**, il package è solo un'etichetta
    B. In una cartella **`modello`**
    C. In una cartella **`package/modello`**
    D. In una cartella **`src/package`**
    E. Nessuna delle precedenti

10. *Quale affermazione su un metodo* **`void`** *è vera?*
    A. Non può avere parametri
    B. Deve terminare con **`return;`**
    C. Può restituire **`null`**
    D. Può usare **`return;`** per uscire prima della fine
    E. Nessuna delle precedenti

11. *Che cosa succede con* **`System.out.println(v[3])`** *dopo* **`int[] v = new int[3]`**?
    A. Il codice non compila
    B. Stampa **`0`**
    C. Lancia **`ArrayIndexOutOfBoundsException`**
    D. Stampa **`null`**
    E. Nessuna delle precedenti

12. *A che cosa serve ridefinire* **`toString()`** *in una classe?*
    A. A confrontare il contenuto di due oggetti
    B. A creare un oggetto partendo da una stringa
    C. A trasformare tutti i campi in **`String`**
    D. A descrivere l'oggetto come testo, per esempio in **`println`**
    E. Nessuna delle precedenti

13. *Che effetto ha il* **`;`** *in* **`while (i < 3); { i++; }`**, *con* **`i`** *che vale 0?*
    A. Nessuno, viene ignorato
    B. Il ciclo ha corpo vuoto, **`i`** non cambia e il programma non termina
    C. Errore di compilazione
    D. Il blocco viene eseguito una volta sola, poi il ciclo termina
    E. Nessuna delle precedenti

14. *Quale affermazione sul metodo* **`main`** *è vera?*
    A. Da Java 25 la forma **`public static void main(String[] args)`** non è più valida
    B. Il file che contiene il **`main`** deve chiamarsi sempre **`Main.java`**
    C. Un programma compilato si avvia con **`java Main.class`**
    D. Una classe senza **`main`** non si può compilare
    E. Nessuna delle precedenti

15. *Hai appena eseguito* **`git commit`**. *Dove si trova ora la modifica?*
    A. Su GitHub, visibile a tutti
    B. Ancora soltanto nell'area di stage
    C. Nel repository locale: per GitHub serve **`git push`**
    D. Soltanto nella working directory
    E. Nessuna delle precedenti

Parte 2 · Che cosa stampa (6 punti, 1,5 a esercizio)

Scrivere l'output esatto, oppure indicare che il codice non compila o che fallisce a runtime.

16. *Che cosa stampa questo codice?*

```
class Articolo {
    String codice;
    double costo;

    Articolo(String codice, double costo) {
        codice = codice;
        this.costo = costo;
    }
}

// nel main
Articolo a = new Articolo("A7", 2.5);
System.out.println(a.codice + " " + a.costo);
```

Output

17. *Che cosa stampa questo codice?*

```
class Salvadanaio {
    int euro;
}

static void versa(Salvadanaio s, int importo) {
    s.euro = s.euro + importo;
    importo = 0;
    s = new Salvadanaio();
    s.euro = 999;
}

// nel main
Salvadanaio m = new Salvadanaio();
int v = 30;
versa(m, v);
System.out.println(m.euro + " " + v);
```

Output

18. *Che cosa stampa questo codice?*

```
String a = "java";
String b = "java";
String c = new String("java");
String d = a;
a = a + "!";
System.out.println(b == c);
System.out.println(b.equals(c));
System.out.println(b == d);
System.out.println(a + " " + d);
```

Output

19. *Che cosa stampa questo codice?*

```
import java.util.Arrays;

// nel main
int x = 10;
x += 2.5;
int[] v = {1, 2, 3};
for (int e : v) {
    e = e * x;
}
System.out.println(x + " " + Arrays.toString(v));
```

Output

Parte 3 · Trova l'errore (4,5 punti, 1,5 a esercizio)

Indicare la riga, se l'errore è di compilazione o di esecuzione, e la correzione.

20. *Dove si trova l'errore?*

```
1  class Cuffia {
2      String marca;
3
4      Cuffia(String marca) {
5          this.marca = marca;
6      }
7  }
8
9  // nel main
10 Cuffia c = new Cuffia();
11 System.out.println(c.marca);
```

Riga _____

☐ Compilazione ☐ Esecuzione

Correzione

21. *Dove si trova l'errore?*

```
1  class Contatore {
2      int valore;
3
4      static void incrementa() {
5          valore++;
6      }
7  }
```

Riga _____

☐ Compilazione ☐ Esecuzione

Correzione

22. *Dove si trova l'errore?*

```
1  // Contatto ha costruttore (String) e getNome()
2  Contatto[] rubrica = new Contatto[3];
3  rubrica[0] = new Contatto("Anna");
4  for (int i = 0; i < rubrica.length; i++) {
5      System.out.println(rubrica[i].getNome());
6  }
```

Riga _____

☐ Compilazione ☐ Esecuzione

Correzione

Parte 4 · Listato (11 punti)

```
class Armadietto {
    private static int creati = 0;
    private int numero;
    private int codice;
    private int tentativi;
    private boolean bloccato;
    private Studente proprietario;

    Armadietto(int numero, int codice) {
        this.numero = numero;
        this.codice = codice;
        creati++;
    }

    void assegna(Studente s) {
        if (!bloccato) {
            proprietario = s;
        }
    }

    boolean apri(int tentativo) {
        if (bloccato) {
            return false;
        }
        if (tentativo == codice) {
            tentativi = 0;
            return true;
        }
        tentativi++;
        if (tentativi == 3) {
            bloccato = true;
        }
        return false;
    }

    static int getCreati() { return creati; }
}
```

```
class Studente {
    private String nome;

    Studente(String nome) { this.nome = nome; }
    String getNome() { return nome; }
    void setNome(String nome) { this.nome = nome; }
}

public class Main {
    public static void main(String[] args) {
        Studente s1 = new Studente("Anna");
        Studente s2 = new Studente("Luca");
        Armadietto a = new Armadietto(1, 111);
        Armadietto b = new Armadietto(2, 222);
        Armadietto c = a;
        a.assegna(s1);
        // PUNTO A
        c.apri(0);
        a.apri(5);
        b.assegna(s2);
        s2.setNome("Marco");
        // PUNTO B
        a.apri(111);
        b.assegna(s1);
        c = b;
        a = null;
        b.apri(1);
        b.apri(2);
        b.apri(3);
        b.assegna(s2);
        // PUNTO C
        System.out.println(c.apri(222));
        System.out.println(Armadietto.getCreati());
    }
}
```

23. **Stato del programma (5 punti).** *Per ogni punto indicato, compilare la tabella.*

	Armadietto numero 1			Armadietto numero 2		
	tentativi	bloccato	proprietario	tentativi	bloccato	proprietario
PUNTO A						
PUNTO B						
PUNTO C						

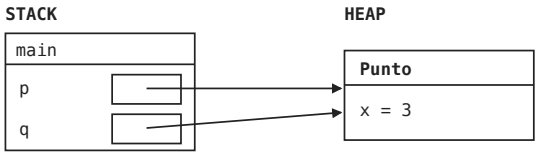
Indicare inoltre che cosa stampa il programma e quale `Armadietto`, al PUNTO C, può essere eliminato dal garbage collector.

Stampa, prima riga _____ seconda riga _____ Armadietto eliminabile _____

24. **Diagramma (3 punti).** *Disegnare stack e heap al PUNTO B: variabili locali del* `main`*, oggetti con i valori dei campi, riferimenti come frecce. Non serve disegnare* `args` *e il campo* `static` `creati`*; i nomi si possono scrivere dentro il campo* `nome`*.*

Esempio di notazione, per:

```
Punto p = new Punto(3);
Punto q = p;
```



25. **Metodo da completare (3 punti).** *Aggiungere ad* `Armadietto` *il metodo seguente: restituisce* `true` *se entrambi gli armadietti hanno un proprietario e si tratta dello stesso oggetto* `Studente`*. Se* `altro` *è* `null` *restituisce* `false`*.*

```
boolean stessoProprietario(Armadietto altro) {

}
```